

## 第 11 章 综合项目

前面十一章的东西都是分开学的。这一章把它们串成一个完整的小项目，从造数据到出结论，走一遍真实的分析流程。

### 11.1 要做的事

**本节任务：**一家小店有一年的销售流水，我们要回答几个问题：全年卖了多少、哪个品类最赚钱、哪个月最旺、有没有异常的记录。

完整流程分五步，每一步都用到前面某一章的知识：

表 11.1：项目的五个步骤和对应章节。

| 步骤   | 用到什么              | 来自哪章  |
|------|-------------------|-------|
| 准备数据 | random 造流水，写成 CSV | 第五、六章 |
| 读取检查 | 读 CSV，查缺失和异常      | 第六、十章 |
| 清洗   | 丢掉或修正坏记录，加派生列     | 第七、十章 |
| 分析   | 分组汇总，排序           | 第十章   |
| 出结果  | 打印结论，画图存盘         | 第十章   |

### 11.2 第一步，准备数据

**本节任务：**造一份带瑕疵的流水数据，存成 CSV。真实数据从来不干净，所以我们故意掺几条问题记录。

```
1 import csv, random
2 from pathlib import Path
3
4 random.seed(42) # 固定种子，每次跑结果一样
5 cats = ["文具", "零食", "饮料"]
6 rows = []
7 for i in range(300):
8     rows.append({
9         "id": i + 1,
10        "month": random.randint(1, 12),
11        "category": random.choice(cats),
12        "amount": random.randint(5, 200),
13    })
```

```
14
15 # 故意掺进三条问题记录
16 rows[10]["amount"] = "" # 缺失
17 rows[50]["amount"] = -30 # 负数
18 rows[80]["category"] = "" # 品类缺失
19
20 with open("sales.csv", "w", encoding="utf-8-sig", newline="") as f:
21     w = csv.DictWriter(f, fieldnames=["id", "month", "category", "amount"])
22     w.writeheader()
23     w.writerows(rows)
24
25 print("已生成", Path("sales.csv").resolve(), "共", len(rows), "条")
```

## 11.3 第二步，读进来先看看

**本节任务：**拿到数据不要急着算，先看清它长什么样、有什么毛病。

```
1 import pandas as pd
2
3 df = pd.read_csv("sales.csv", encoding="utf-8-sig")
4 print("形状: ", df.shape)
5 print(df.head(3))
6 print("\n每列缺失数: ")
7 print(df.isna().sum())
8 print("\n金额列的摘要: ")
9 print(df["amount"].describe())
```

摘要里如果最小值是负数，就说明有异常记录。这一步的目的就是把问题找出来，而不是直接进入计算。

## 11.4 第三步，清洗

**本节任务：**把坏记录处理掉，再加一列方便后面分析。

```
1 before = len(df)
2
3 df = df.dropna(subset=["amount", "category"]) # 丢掉关键列缺失的行
4 df = df[df["amount"] > 0] # 丢掉金额非正的行
5 df["amount"] = df["amount"].astype(int) # 读进来可能是小数，转成整数
```

```
6
7 def level_of(x):
8     """按金额分档，用第四章的函数写法"""
9     if x >= 150:
10         return "大额"
11     if x >= 50:
12         return "中额"
13     return "小额"
14
15 df["level"] = df["amount"].apply(level_of) # 用函数生成新列
16
17 print(f"清洗前 {before} 条，清洗后 {len(df)} 条，丢掉 {before - len(df)} 条"
18       )
19 print(df.head(3))
```

apply 把一个函数套用到整列的每个值上，这是把自己写的函数用到表格上的常见 ★核心办法，第四章学的函数在这里派上用场。

## 11.5 第四步，分析

本节任务：回答一开始提出的几个问题。

```
1 print("全年总额：", df["amount"].sum())
2 print("笔均：", round(df["amount"].mean(), 1))
3
4 print("\n各品类总额：")
5 by_cat = df.groupby("category")["amount"].sum().sort_values(ascending=
6     False)
7 print(by_cat)
8
9 print("\n最赚钱的品类：", by_cat.index[0])
10
11 print("\n各月总额：")
12 by_month = df.groupby("month")["amount"].sum()
13 print("最旺的月份：", by_month.idxmax(), "月，", by_month.max(), "元")
14
15
16 print("\n各档次笔数：")
17 print(df["level"].value_counts())
18
19
20 print("\n品类与档次交叉：")
21 print(pd.crosstab(df["category"], df["level"]))
```

## 11.6 第五步，出图

**本节任务：**把两个主要结论画成图存下来。

```
1 import matplotlib
2 matplotlib.use("Agg")
3 import matplotlib.pyplot as plt
4
5 fig, axes = plt.subplots(1, 2, figsize=(11, 4))
6
7 by_cat.plot(kind="bar", ax=axes[0], title="Sales by Category")
8 by_month.plot(kind="line", marker="o", ax=axes[1], title="Sales by Month")
9 axes[1].set_xlabel("month")
10
11 plt.tight_layout()
12 plt.savefig("report.png", dpi=120)
13 print("图已存成 report.png")
```

到这里项目就完成了。从一堆原始记录出发，经过检查、清洗、分析、可视化，得到了能说给别人听的结论。

## 11.7 这个流程为什么这么排

**本节任务：**把套路记住，换个数据也能照着做。

五步的顺序不是随便定的。

先看后算这一点最重要。**没检查就直接算，等于在错误的数据上做正确的运算**，结果看着像模像样，实际全错。前面那条负数记录如果不清掉，总额和平均都会被拉低，而你根本不会发现。

清洗要留痕。上面打印了清洗前后的条数，丢掉多少心里有数。如果丢掉了一大半，那多半是清洗规则写得太狠，得回头检查。

分析从粗到细。先看总量，再按维度拆开，最后交叉看。一上来就做复杂的交叉分析，容易在细节里迷路。

结论要能一句话说清。全年多少、哪个品类最好、哪个月最旺，这些是要给人看的。图是帮人快速看懂的工具，不是目的。

换成别的数据，比如学生成绩、网站访问量、天气记录，这套流程照样能用。数据分析的日常工作大部分就是这五步的反复。

## 11.8 学完这十二章之后

到这里，这份教程就结束了。回头看一下走过的路。

前五章打地基，变量和类型、列表和字典、分支和循环、函数，这些是任何程序的骨架。中间三章让程序能用起来，借用现成的库、把数据存进文件、出错时不崩溃。第八章用类把数据和操作打包。最后三章转向数据，数组、表格、完整项目。

接下来往哪走，看你想做什么。想做数据分析，继续深入 pandas 和统计知识。想做网站后端，去学 Flask 或 FastAPI。想做自动化脚本，标准库里的 os 和 subprocess 值得细看。想往人工智能方向走，这一章的数组和表格正好是那边的入口，深度学习框架里的张量就是数组的延伸。

不管往哪走，方法都一样，找一个具体的小目标，动手做，卡住了就查，做完再回头补原理。这份教程每一章都是这么安排的，往后也这么学就行。

### 本章知识点

- **五步流程** —— 准备数据、读取检查、清洗、分析、出结果
- **先看后算** —— 没检查就算，等于在错误的数据上做正确的运算
- **清洗留痕** —— 打印清洗前后的条数，丢太多说明规则写狠了
- **apply** —— 把自己写的函数套用到整列的每个值上
- **value\_counts** —— 数一列里每个取值出现多少次
- **crosstab** —— 交叉统计两个维度
- **idxmax** —— 找出最大值对应的标签，比如哪个月最旺
- **固定随机种子** —— 让每次运行结果一样，便于核对
- **从粗到细** —— 先看总量，再按维度拆，最后交叉看